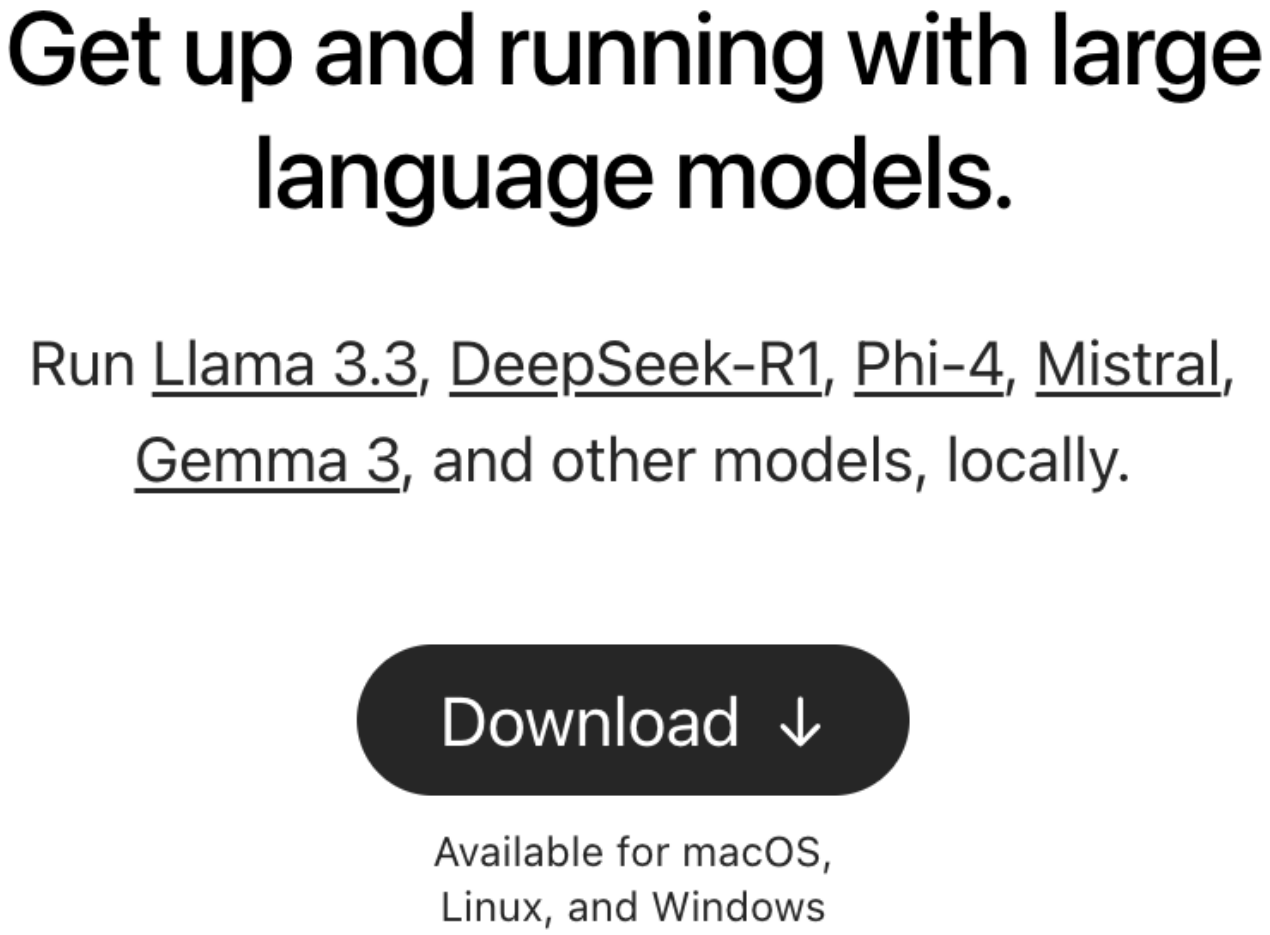
六、本地构建大模型服务

在之前章节我们用美团的客服问答信息，整合阿里云百炼的通义千问大模型，构建起了一个RAG应用。但是，你可能会发现一个问题。我们这个RAG应用虽然也算实现了只能问答的功能，但是，需要将美团的客服问答信息上传到阿里的大模型上，这是不是就造成了数据安全的问题？当然在我们这个案例中，问题可能还不太严重，因为我们使用的数据就是美团对外公开的数据。但是，如果是一些涉及到企业机密的信息呢？
因此，很多企业在构建基于AI大模型的应用时，都会采用本地部署大模型的方式。这样即可以减少访问开源大模型的网络和Token成本，也可以防止数据泄露。
这一章节，我们就来搭建一个本地大模型服务体系。首先你要清楚，我们能够搭建本地的大模型服务，这得益于现在很多大模型的研发者都会将辛苦训练出来的大模型对外开源，这样我们才能接触到这些预训练模型。然后，现在构架本地大模型的方式和工具也有很多，这里只是分享一套最为典型的大模型应用体系。

- 1、使用Ollama本地运行DeepSeek
- 2、本地应用调用Ollama模型
- 3、One-API部署本地大模型网关
- 4、FastGPT搭建本地知识库

1、 使用Ollama本地运行DeepSeek

Ollama是一个可以在本地运行的大模型服务平台。官网地址：https://ollama.com/



进入下载页面，选择对应平台的应用程序，安装即可。安装完成后，控制台输入ollama -v，可以查看ollama的版本，验证ollama是否安装成功。

```
% ollama -v
ollama version is 0.5.4
```

然后在官网点击models，就可以查看Olalma平台上支持的开源大模型。其中包含了很多世界前沿的企业开放的优秀大模型。

gemma3

The current strongest model that fits on a single GPU.

Google

1b 4b 12b 27b

101.2K Pulls 9 Tags Updated 22 hours ago

---

qwq

QwQ is the reasoning model of the Qwen series.

阿里

tools 32b

881.1K Pulls 8 Tags Updated 7 days ago

---

deepseek-r1

DeepSeek's first-generation of reasoning models with comparable performance to OpenAI-o1, including six dense models distilled from DeepSeek-R1 based on Llama and Qwen.

深度求索

1.5b 7b 8b 14b 32b 70b 671b

25.8M Pulls 29 Tags Updated 4 weeks ago

---

llama3.3

New state of the art 70B model. Llama 3.3 70B offers similar performance compared to the Llama 3.1 405B model.

Meta

tools 70b

1.5M Pulls 14 Tags Updated 3 months ago

选择一个大模型，进入模型详情页，就可以看到部署的指令。这里以deepseek-r1为例：

Readme

这里1B就表示1 Billion十亿参数。通常参数越多，模型推理能力越强。但是相应的，运行模型所需要的资源也就越多。
这里我们可以使用参数量最小的deepseek-r1:1.5b作为本地测试。执行 ollama run deepseek-r1:1.5b。Ollama就会自行下载模型到本地目录，并启动对应的服务。

> 如果是windows系统，Ollama会默认把模型文件放在C:\Users\用户名.ollama\models\目录下。如果需要在其他目录下存放模型，可以在ollama run指令后面加上--model-dir参数，指定模型存放的目录。
>
> ```
> % ollama run deepseek-r1:1.5b --model-dir /Users/zhangjun/ollama/models
> ```
>
> 或者也可以配置环境变量 OLLAMA_MODELS，通义定制模型文件存放的目录。

```
(base) [redacted] ~ % ollama list
NAME                        ID              SIZE      MODIFIED
deepseek-r1:1.5b            a42b25d8c10a    1.1 GB    5 weeks ago
my-custom-model:latest      3299599b2562    2.5 GB    2 months ago
(base) [redacted] ~ % ollama run deepseek-r1:1.5b
>>> 你是谁
<think>

</think>

您好！我是由中国的深度求索（DeepSeek）公司开发的智能助手DeepSeek-R1。如您有任何任何问题，我会尽我所能为您提供帮助。

>>> Send a message (/? for help)
```

这样就部署好了一个本地的大模型服务。我们就可以使用ollama的命令行工具来测试这个模型了。

2、本地应用调用Ollama模型

1-调用Ollama的聊天模型

基于LangChain良好的整体设计，我们也可以像之前使用其他模型一样使用Ollama的模型。

```
from langchain_community.chat_models import ChatOllama

llm = ChatOllama(model="deepseek-r1:1.5b",base_url="http://localhost:11434")
llm.invoke("你是谁？能帮我解决什么问题？")
```

> 参数方面，由于Ollama是本地部署的，所以自然不需要api_key进行计费了。另外，部署地址不稳定，所以需要指定base_url。

另外，其实Ollama也是兼容OpenAI的接口协议的，所以，我们也可以使用OpenAI的接口来调用Ollama的模型。

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="deepseek-
r1:1.5b",base_url="http://localhost:11434/v1",openai_api_key="123")
llm.invoke("你是谁？能帮我解决什么问题？")
```

> 注意访问地址是有不同的。

接下来，聊天记录、本地工具、链式调用等操作，与之前使用LangChain的方式基本没有什么区别。你都可以动手试试 。

2- 调用Ollama的Enbedding模型

Ollama平台上不光包含大语言模型，同时也有很多Embedding向量化模型。

All | Embedding | Vision | Tools | Popular

nomic-embed-text

A high-performing open embedding model with a large token context window.

embedding

18.8M Pulls 3 Tags Updated 12 months ago

mxbai-embed-large

State-of-the-art large embedding model from mixedbread.ai

embedding 335m

1.8M Pulls 4 Tags Updated 10 months ago

这些向量化模型只需要使用ollama pull nomic-embed-text指令拉取，不需要运行。接下来就可以使用LangChain中提供的OllamaEmbedding类来调用这些模型了。

```
#安装ollama依赖
!pip install langchain_ollama
```

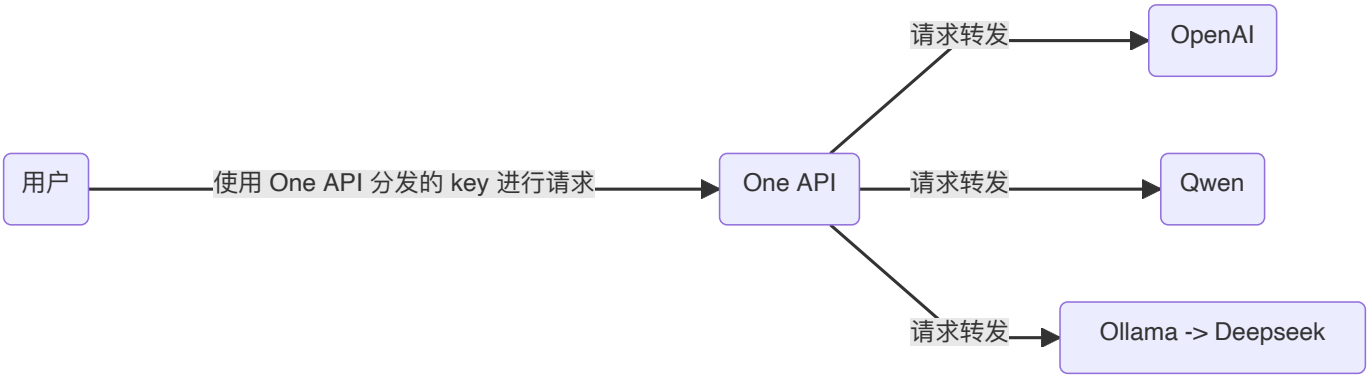
```
from langchain_ollama import OllamaEmbeddings
embedding_model = OllamaEmbeddings(model="nomic-embed-
text",base_url="http://localhost:11434")

embedding = embedding_model.embed_query("你好")
print(len(embedding))
```

> 从这个案例也能看到，Ollama的一些核心功能正在从langchain_community逐步迁移到langchain_ollama中。这些依赖关系的调整，也是未来版本经常会出现的事情。

接下俩，同样可以用这个向量化模型逐步构建RAG应用。这和我们之前演练的案例基本上是一样的。
对于RAG应用来说，Embedding和LLM其实并不一定需要采用同一个模型，不同模型组合，有些时候也可能带来更好的效果。

3、One-API搭建本地大模型网关

如果你的本地应用稍微复杂一点，就可能需要多个大模型协同工作了。这时，One-API就派上了用场。
One-API是一个本地大模型网关。One-API提供了一套遵循OpenAI标准的统一接入API，而后端可以把请求转发给多个不同的大模型产品。



One-API的安装和部署，可以参考：https://github.com/songquanpeng/one-api 。 重点关注他支持的大模型产品。

1-Docker部署

One-API有多种部署方式。最简单的是使用Docker部署。

```
#并发量小，用SQLlite
docker run --name one-api -d --restart always -p 3000:3000 -e TZ=Asia/Shanghai -v
/Users/roykingw/docker/one-api:/data justsong/one-api
#并发量大，用MySQL 。 添加 `-e SQL_DSN="root:123456@tcp(localhost:3306)/oneapi"` 指向MySQL
```

启动完成后，访问 localhost:3000，就可以进入One-API的页面

登录时，默认的用户名是root，密码是123456。登录后，会建议你修改默认密码。这时，可以按照页面指示，修改下密码。

2-配置渠道

渠道是One-API的核心功能。一个渠道表示一个后端的大模型服务。 这些大模型服务，可以是本地的Ollama服务，也可以是互联网上的大模型服务。例如，下图是配置国内的智谱AI，再填入智谱AI的key，就可以通过这个渠道与访问智谱AI了。

当然，而已可以配置自己本地的Ollama模型。配置时，模型中填入在Ollama中部署的模型。秘钥随便填写一个即可。
如果是用Docker部署的One-API镜像服务，而Ollama部署在宿主机时，需要注意配置一个可以从One-API镜像访问宿主机的代理地址。 http://host.docker.internal:11434 （mac下可以是 http://docker.for.mac.host.internal:11434 ）。这个地址是docker支持的一个默认的可以访问宿主机的域名。

3-配置令牌

令牌就是访问One-API的身份凭证。其作用就和其他大模型平台的api-key是一样的。

4-通过One-API调用大模型

One-API提供了一套兼容OpenAI规范的访问接口，所以，我们可以直接使用OpenAI的相关客户端访问 One-API

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
        model="glm-3-turbo", # one-api中支持的model。默认会找这个model的第一个渠道进行转发
        openai_api_key="sk-3FWIdr7ji3lHCz4sEcAe71C5F46f4f4e971aCe980736E09f", #one-api中分配的令牌
        base_url="http://localhost:3000/v1") #one-api的访问地址

llm.invoke("你是谁？能帮我解决什么问题？")
```

通过One-API，我们就可以在本地快速构建起能够支持多个不同大模型的大模型服务体系，可以灵活构建本地的大模型应用。例如，对于一些不涉及到机密数据的问题，可以调用开源大模型，获得更稳定的服务。而对于一些涉及到了公司机密数据的问题，就可以调用本地部署的大模型，保证数据安全。

当然，如何区分用户的问题是否涉及到公司机密数据呢？这就是典型的意图识别的任务，通常通过合理定制提示词就可以做到。

另外，One-APi还有很多相关的项目。在github主页的下面就能看到。

相关项目

- FastGPT: 基于 LLM 大语言模型的知识库问答系统
- ChatGPT Next Web: 一键拥有你自己的跨平台 ChatGPT 应用
- VChart: 不只是开箱即用的多端图表库，更是生动灵活的数据故事讲述者。
- VMind: 不仅自动，还很智能。开源智能可视化解决方案。

这其中，FastGPT算是最有价值的项目了。通过FastGPT，我们可以在Weby页面上快速构建一个RAG智能问答系统。

4、FastGPT搭建本地知识库

FastGPT项目地址： https://github.com/labring/FastGPT

默认情况下，FastGPT只配置了GPT的模型，如果需要接入其他模型，可以通过整合one-api进行部署，参见文档：https://doc.tryfastgpt.ai/docs/development/modelconfig/one-api/ . FastGPT有多种部署模式，这里采用基于Docker进行本地部署。

接下来，我们就用FastGPT+本地Ollama，快速部署一个RAG智能问答系统。

1-下载docker-compose.yml文件

FastGPT提供了一套基于docker-compose的快速部署方式。需要下载两个文件， config.json(fastGPT自己的配置文件)和docker-compose.yml(docker-compose部署文件)

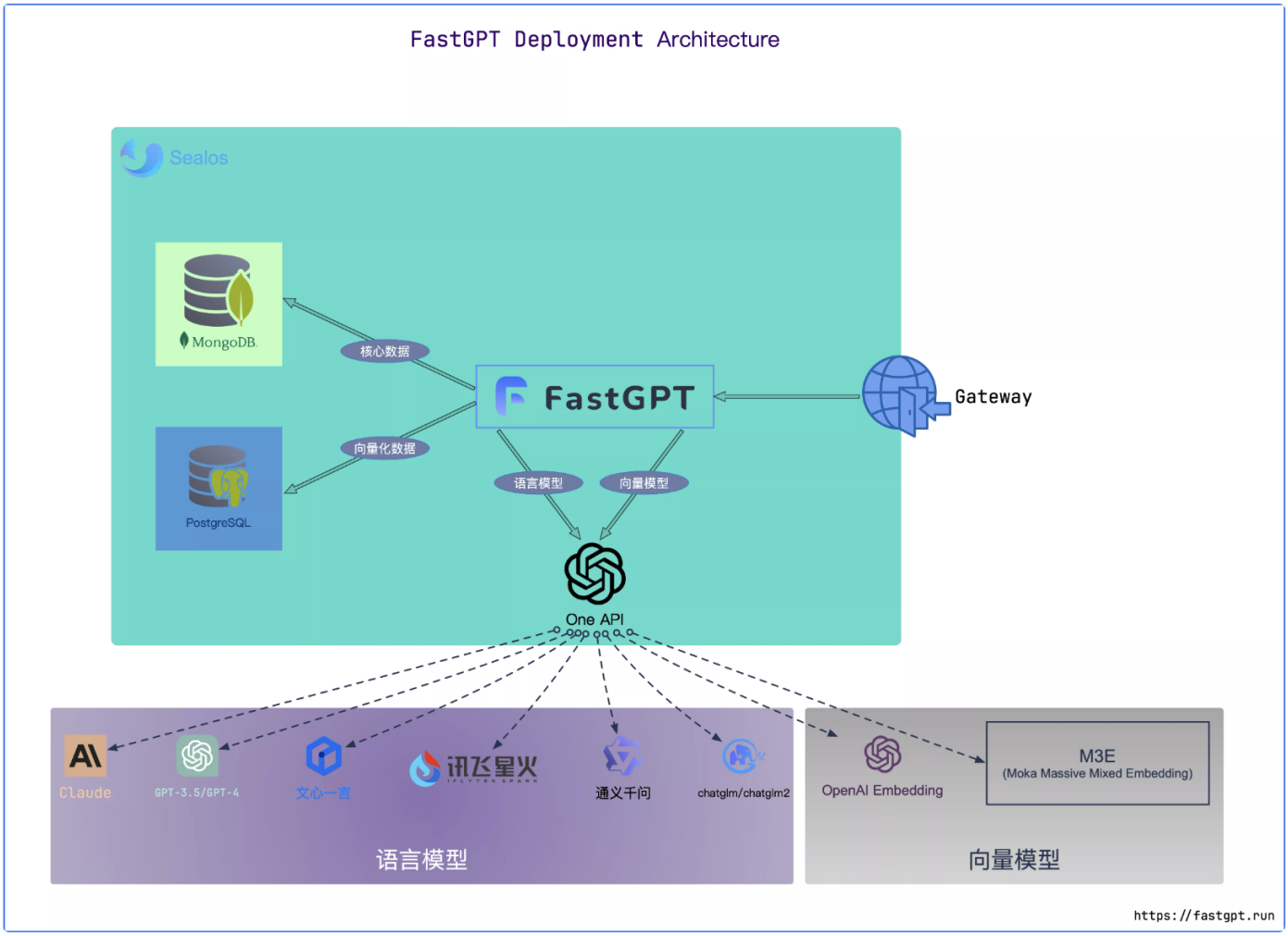
```
mkdir fastgpt
cd fastgpt
curl -O
https://raw.githubusercontent.com/labring/FastGPT/main/projects/app/data/config.json

# pgvector 版本(测试推荐，简单快捷)
curl -o docker-compose.yml
https://raw.githubusercontent.com/labring/FastGPT/main/files/docker/docker-compose-
pgvector.yml
```

2-使用docker-compsoe直接启动镜像

docker-compose.yml文件中配置了fastGPT所有相关的服务。默认情况下，不做任何修改，我们就可以直接启动。

```
docker-compose up -d
```

当然，如果你熟悉docker-compose的配置的haul，也可以配置成自己的服务



- MongoDB: 用于存储除了向量外的其他各类数据
- PostgreSQL/Milvus: 存储向量数据
- OneAPI：聚合各类AI，支持多模型调用

3-配置基础环境

首先：需要配置One-API中访问大模型的渠道和令牌。
这个配置方式和之前使用One-API没有什么区别。只是，默认情况下，FastGPT把One-API服务部署到了3001端口。 原来的3000端口由fastGPT自己使用。
然后：我们需要修改docker-compose.yml文件。
主要是将FastGPT的CHAT_API_KEY修改为One-API中生成的令牌。然后重启fastGPT

!

接下来：需要配置FastGPT自己使用的大模型
在config.json文件中配置了FastGPT可以访问的大模型。这里需要定制的是llmModels大语言模型和vectorModels向量化模型。llmModels大语言模型，可以配置成本地在Ollama上运行的那些模型。例如：

```
//语言大模型
"llmModels": [
    {
      "model": "deepseek-r1:1.5b", // 本地Ollama运行的大模型
      "name": "deepseek-r1:1.5b", // 模型别名
      "avatar": "/imgs/model/openai.svg", // 模型的logo
      "maxContext": 125000, // 最大上下文
      "maxResponse": 16000, // 最大回复
      "quoteMaxToken": 120000, // 最大引用内容
      "maxTemperature": 1.2, // 最大温度
      "charsPointsPrice": 0, // n积分/1k token（商业版）
      "censor": false, // 是否开启敏感校验（商业版）
      "vision": true, // 是否支持图片输入
      "datasetProcess": true, // 是否设置为文本理解模型（QA），务必保证至少有一个为true，否则知识库会报错
      "usedInClassify": true, // 是否用于问题分类（务必保证至少有一个为true）
      "usedInExtractFields": true, // 是否用于内容提取（务必保证至少有一个为true）
      "usedInToolCall": true, // 是否用于工具调用（务必保证至少有一个为true）
      "usedInQueryExtension": true, // 是否用于问题优化（务必保证至少有一个为true）
      "toolChoice": true, // 是否支持工具选择（分类，内容提取，工具调用会用到。）
      "functionCall": false, // 是否支持函数调用（分类，内容提取，工具调用会用到。会优先使用toolChoice，如果为false，则使用 functionCall，如果仍为 false，则使用提示词模式）
      "customCQPrompt": "", // 自定义文本分类提示词（不支持工具和函数调用的模型
      "customExtractPrompt": "", // 自定义内容提取提示词
      "defaultSystemChatPrompt": "", // 对话默认携带的系统提示词
      "defaultConfig": {}, // 请求API时，挟带一些默认配置（比如 GLM4 的 top_p）
      "fieldMap": {} // 字段映射（o1 模型需要把 max_tokens 映射为 max_completion_tokens）
    },
    .......
}
```

向量化模型部分，可以采用M3E模型。这里使用Docker直接运行M3E镜像：

```
# CPU运行
docker run -d --name m3e -p 6008:6008 registry.cn-hangzhou.aliyuncs.com/fastgpt_docker/m3e-large-api:latest
# GPU运行可以加上参数 --gpus
```

接下来：需要在One-API中配置M3E模型的访问渠道

配置完成后，修改config.json文件当中的向量化模型。 增加一个m3e的选项

```
//向量化模型
"vectorModels": [
     {
      "model": "m3e",
      "name": "M3E（测试使用）",
      "price": 0.1,
      "defaultToken": 500,
      "maxToken": 1800
    },
  .....
  ]
```

最后，重启fastGPT镜像

```
docker-compose down
docker-compose up -d
```

4-fastGPT中配置知识库

fastGPT默认访问端口3000。默认用户名root，密码是docker-compose.yml文件中进行配置的。
选择知识库，新建一个通用知识库，索引模型选择m3e,文本模型选择deepseek-r1:1.5b(ollama本地运行的大模型)
创建完成后，选择新建，导入手动数据集。

然后，上传本地知识库文件。这里用我们之前用过的meituan-questions.txt为例
上传之后，fastGPT就会自动完成rag的index索引工作。将文本内容进行拆分，转成文本向量后，保存到向量数据库中。全部结束后，就可以对数据集进行检索测试。

5-fastGPT中创建RAG应用

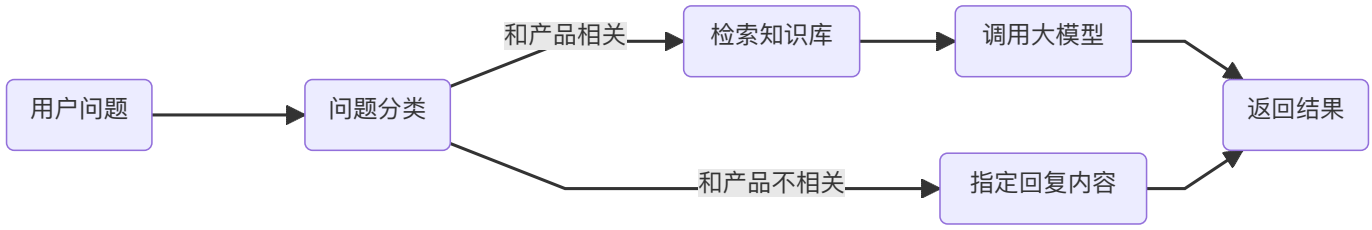
进入fastGPT的工作台，新建应用，选择知识库+对话引导应用。创建完成后，进入应用，选择AI模型和关联知识库，就可以用知识库中的内容，跟AI进行愉快的聊天了。

5、使用总结

在这一章节的演练过程中，我们不只是搭建了本地大模型，其实是围绕Ollama搭建了一整套本地大模型的基础服务。这里面，甚至可以看到一些传统服务架构的影子：

如何把单行工具用好，这是业界很多人都在不断探索的一个目标。这些开源的工具和框架，就是最为成熟的业界经验。
例如，有了one-api作为网关，我们可以很容易的在同一个平台上访问多个大模型。这意味着我们完全可以结合多个大模型共同来处理一些复杂的问题。

另外，fastGPT中还提供了工作流，这也是结合大模型的一种使用经验。这些工作流并不难理解，例如fastGPT中一个默认的工作流就是先判断用户的问题和产品是否相关，不相关直接拒绝(例如询问今天天气怎么样)。相关问题再去调用大模型(例如如何发起退款，退款多长时间到账等)。



这个过程中，“问题分类”和"调用大模型"这两个步骤，都可以交由大模型完成。而基于任务的难度和性质不同，将他们交由不同的大模型处理也是不错的选择。 例如"问题分类"步骤，通常并不涉及到企业内部的机密信息，所以，直接交由互联网上现成的大模型处理，可以直接节省很多计算资源。而后续"调用大模型"步骤，放在本地大模型处理，这样就可以保证数据安全。另外，由于在这个步骤需要处理的任务其实是比较单一的，所以，针对这个步骤，对大模型进行微调，形成自己的行业大模型也是一个非常好的选择。